

Bare-Metal Programming for NXP FRDM-IMX91

Report / Practical Guide

Shreekar Mane (B23CS1069)

Tavishi Srivastava (B23CS1101)

Yadav Karan Subhashchandra (B23EE1103)

Nallagonda Dharshan (B23EE1044)

April 17, 2026

Abstract

This manual serves as a comprehensive, practical guide to bare-metal programming on the NXP FRDM-i.MX91 development board. It explores the fundamental architecture of the AArch64 Cortex-A CPU and the execution flow from Boot ROM to U-Boot, and from U-Boot to the custom Application binary.

By referencing the official NXP hardware documentation (IMX91RM and UM12262), readers will learn how to parse memory maps, configure a custom cross-compilation build system, directly manipulate memory-mapped peripheral registers, and handle hardware-level precise timing.

The manual culminates in two real-world applications. First, the development of a simple hardware driver for an HC-SR04 ultrasonic sonar sensor (used as example in this report/manual), complete with digital signal processing techniques to minimize reading fluctuations. Second, the custom-built, preemptive Real-Time Operating System (we gave it name littleOS) from scratch upon this bare-metal framework. littleOS showcases advanced embedded concepts including dynamic scheduling (Round Robin, Priority, EDF), ARMv8 hardware atomics for thread-safe peripheral access, and a passive TCP interceptor via an ESP8266 module to seamlessly integrate the board as an IoT edge node with an external Python Flask Command Hub.

You will find the complete source code for both the bare-metal framework and littleOS here: https://github.com/shreekar2005/BareMetal_FRDM_IMX91.

Contents

1	Architecture and Boot Flow	2
1.1	The Execution Flow: Boot ROM → U-Boot → Bare-Metal	2
1.2	Why We Need U-Boot	2
1.3	What are eMMC Partitions	3
2	The Bare-Metal Environment Setup	4
2.1	Preparing the Host Machine (Ubuntu Toolchains)	4
2.2	Serial Communication: Picocom and the Baud Rate	4
2.3	Deployment Method	4
3	The Build System	6
3.1	The Linker Script (.ld), Assembly code (start.S and vector.S)	6
3.2	Writing the Makefile for AArch64 Cross-Compilation	7
4	SOFTWARE DEVELOPMENT KIT (SDK)	8
4.1	Input/Output Multiplexer (IOMUX)	8
4.2	Generic Interrupt Controller (GIC)	9
4.3	General-Purpose Input/Output (GPIOs)	10
4.4	Low-Power Universal Asynchronous Receiver-Transmitter (LPUART) . .	12
4.5	System Counter (SYS_CTR)	14
5	Interfacing with the Physical World: HC-SR04 Sonar	15
5.1	Hardware Wiring	15
5.2	Steps to Run	15

Chapter 1

Architecture and Boot Flow

1.1 The Execution Flow: Boot ROM → U-Boot → Bare-Metal

When the NXP FRDM-i.MX91 board is powered on, the execution of software happens in a strictly phased pipeline. The AArch64 Cortex-A core does not immediately start executing our custom bare-metal code. Instead, it follows this sequence:

1. **Boot ROM (Primary Bootloader):** Hardcoded into the silicon by NXP. It reads the SW1 DIP switches (or internal eFuses) to determine the boot device (eMMC, SD Card, or Serial). It initializes basic internal RAM and loads the secondary bootloader.
2. **U-Boot (Secondary Bootloader like grub):** A highly capable open-source bootloader. It initializes the complex external DDR memory, sets up the central clock tree and loads device drivers for USB and SD/eMMC interfaces. U-Boot provides the interactive `=>` prompt over the serial console.
3. **Bare-Metal Application:** Using U-Boot commands (like `fatload`), we copy our compiled `.bin` file from a storage medium directly into the DDR RAM at address `0x80000000`. We then use the `go 0x80000000` command to forcefully hand over the CPU's Program Counter to our application.

1.2 Why We Need U-Boot

In a pure bare-metal environment, one might ask why we do not bypass U-Boot entirely. The answer lies in hardware complexity. Writing a DDR memory controller initialization sequence from scratch requires thousands of lines of highly specific configuration code and precise timing calibration. Furthermore, reading a FAT32 filesystem from a USB drive requires complex USB Host Controller drivers.

By allowing U-Boot to run first, it handles the complex hardware initialization. We essentially use U-Boot as a launchpad. It sets up the 24MHz external oscillator, configures the LPUART for debugging, and initializes the massive DDR RAM so our bare-metal code has immediate, stable access to memory.

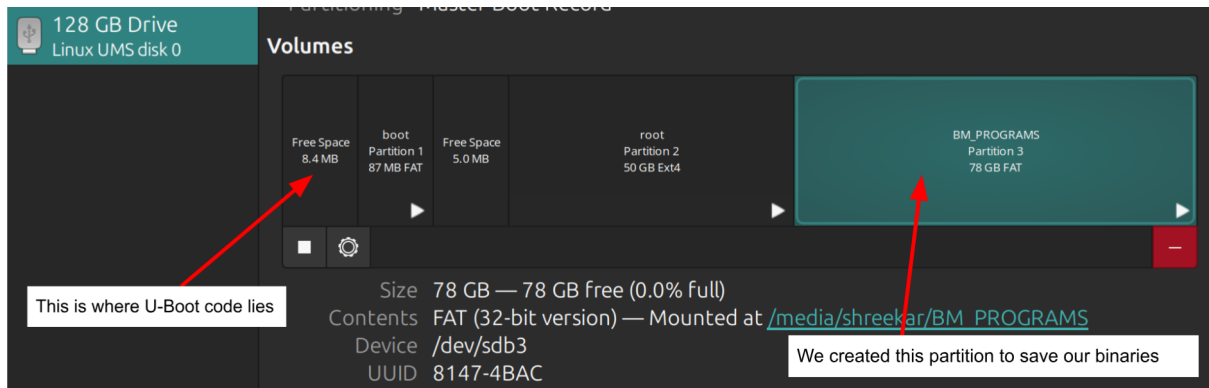


Figure 1.1: How microSD partition look like

1.3 What are eMMC Partitions

Embedded multimedia cards (eMMC) and SD cards are typically divided into physical hardware partitions and software logic partitions. U-Boot relies on specific partition layouts to locate boot files and user data.

When deploying our code via U-Boot Mass Storage (UMS), you will notice partition targeting like `mmc 1:3` or `mmc 0:3`

`mmc 0`: Refers to MMC interface 0 (the onboard eMMC chip)

`mmc 1`: Refers to MMC interface 1 (the external microSD card)

NOTE: We formatted our microSD card using NXP image (you can get that on their website), so that we can keep eMMC as it is (as we don't own NXP-FRDM-board) (Figure 1.1).

- `mmc 0:1`: eMMC partition 1 : Holds linux binaries.
- `mmc 0:2`: eMMC partition 2 : Holds linux root directory.
- `mmc 0:3`: Does not exist by default (but you can create it using laptop).
- `mmc 1:3`: microSD partition 3 : formatted by us as FAT32 to hold our binaries.

By utilizing U-Boot's `fatload` command, we can easily pull our `.bin` files from FAT32 `mmc 1:3` partition directly into system RAM.

Chapter 2

The Bare-Metal Environment Setup

2.1 Preparing the Host Machine (Ubuntu Toolchains)

Because the FRDM-i.MX91 utilizes an AArch64 (64-bit ARM) architecture, we cannot compile the code using a standard x86_64 GCC compiler. We must use a cross-compiler. For Ubuntu, setting this up is straightforward.

Open your terminal and install the GNU toolchain for AArch64:

```
sudo apt update
sudo apt install gcc-aarch64-linux-gnu binutils-aarch64-linux-gnu
```

Verify the installation by checking the compiler version:

```
aarch64-linux-gnu-gcc --version
```

2.2 Serial Communication: Picocom and the Baud Rate

To interact with U-Boot and view the **serial debug output** of our bare-metal applications, we must establish a serial connection over the board's Debug USB port. The standard communication speed (Baud Rate) is **115200 bps**.

We utilize **picocom** for its lightweight nature. Start the serial console with:

```
sudo picocom -b 115200 /dev/ttyACM0
```

2.3 Deployment Method

Our build system uses the U-Boot Mass Storage (UMS) deployment methodology to get the **.bin** file from your Ubuntu host into the board's RAM.

This method temporarily turns the board's storage (SD or eMMC) into a flash drive readable by your PC.

1. At the U-Boot prompt, execute: **ums 0 mmc 1:3** (for SD Card) or **ums 0 mmc 0** (for eMMC).
2. On your Ubuntu machine, copy the generated **.bin** to the mounted drive.

3. Eject the drive from your OS then U-Boot terminal automatically exit UMS mode.
4. Load and run the binary. For example, if using the SD Card:

```
fatload mmc 1:3 0x80000000 app.bin && dcache flush && icalche  
flush && go 0x80000000
```

Note on Caches: The commands `dcache flush` and `icalche flush` are critical. They ensure that the binary written to DDR RAM is properly synchronized with the CPU's internal L1/L2 caches before execution begins.

Chapter 3

The Build System

3.1 The Linker Script (`.ld`), Assembly code (`start.S` and `vector.S`)

A bare-metal application lacks an operating system loader to map memory. We must manually dictate the physical memory layout using a **Linker Script** (`linker.ld`).

```
ENTRY(_start)
SECTIONS {
    . = 0x80000000;
    .text : { *(.text.start) *(.text*) }
    . = ALIGN(16);
    .rodata : { *(.rodata*) }
    . = ALIGN(16);
    .data : { *(.data*) }
    . = ALIGN(16);
    .bss : { __bss_start = .; *(.bss*) *(COMMON) . = ALIGN(16);
        __bss_end = .; }
}
```

The script sets the base address to `0x80000000`, exactly where U-Boot loads our binary. It organizes the code into executable instructions (`.text`), read-only constants (`.rodata`), initialized variables (`.data`), and uninitialized variables (`.bss`).

`start.S`: The Entry Point

The CPU enters our program at the `_start` label. Because we are borrowing hardware states from U-Boot, our `start.S` carefully backs up U-Boot's stack pointer and registers. It then assigns a dedicated, safe stack pointer for our C code (`0x81000000`), zeroes out the `.bss` section to prevent garbage variable values, and branches to `main()`. Upon `main()` returning, it restores U-Boot's state to allow for a clean exit back to the U-Boot console.

`vector.S`: The Exception Table

In AArch64, handling hardware interrupts and CPU faults requires a strictly formatted Exception Vector Table. `vector.S` maps a 2048-byte grid (`.align 11`) containing 16 entry points.

Because our CPU runs at Exception Level 2 (EL2) utilizing the SP_EL2 stack pointer, hardware IRQs from peripherals like the UART or Sonar are routed specifically to **Block 2, Offset +0x280**.

For critical system failures (like memory aborts), the `common_trap` block serves as an autopsy room:

```
common_trap:
    msr daifset, #2          // Disable further IRQs
    ldr x4, =0x80200000      // Switch to emergency stack
    mov sp, x4
    mrs x0, esr_el2          // Save Exception Syndrome
    mrs x1, elr_el2          // Save Exception Link Register
    mrs x2, far_el2          // Save Fault Address
halt_cpu:
    wfi                     // Put CPU to deep sleep
    b halt_cpu
```

3.2 Writing the Makefile for AArch64 Cross-Compilation

Our master Makefile orchestrates the compilation of both our core application and shared hardware drivers.

```
CC = aarch64-linux-gnu-gcc
LD = aarch64-linux-gnu-ld
OBJCOPY = aarch64-linux-gnu-objcopy

CFLAGS = -g -c -nostdlib -ffreestanding -O0 -mno-outline-atomics
```

The flags `-nostdlib` and `-ffreestanding` are the core of bare-metal compilation. They explicitly instruct GCC not to link the standard Linux C library (`glibc`) or the standard startup files (`crt0`), as we have provided our own `start.S` and do not have an underlying OS to handle standard syscalls like `malloc` or `printf`.

The Makefile compiles all source files into `.o` object files, links them according to `linker.ld` to generate an `.elf` file for debugging, and finally uses `objcopy -O binary` to strip away all ELF metadata. This yields a raw, pure `.bin` file that the AArch64 CPU can execute blindly from RAM.

Chapter 4

SOFTWARE DEVELOPMENT KIT (SDK)

NXP FRDM i.MX91 board, unlike STM32, does not come with a pre-packaged, comprehensive Software Development Kit (SDK) or Hardware Abstraction Layer (HAL) by default. To bridge this gap, we engineered a custom, lightweight, and highly robust SDK from scratch. By meticulously studying the i.MX91 Reference Manual, we mapped the memory addresses of critical peripherals and abstracted complex bit-level manipulations into intuitive, reusable C functions.

We will dive deeper into the SDKs for individual peripherals in the coming sections.

4.1 Input/Output Multiplexer (IOMUX)

Let us first look into the distinction between a "pad" (the internal electrical contact on the silicon die) and a "pin" (the external physical connector). Because the i.MX91 processor features significantly more internal peripheral functions than available physical pads, multiple internal peripherals might be connected to the same physical pin. The IOMUX helps by dynamically routing specific internal signals to designated external pads.

The `IOMUX.h` file manages the IOMUXC registers, abstracting the low-level complexity of this multiplexing configuration. Within our custom SDK, this module provides a targeted interface for routing the physical pads, specifically for the P11 Expansion Header. It defines the necessary base addresses, Pad Mux Registers, and Daisy Chain registers. While MUX registers map a pad to a peripheral function, Daisy Chain registers are critical for resolving input conflicts by explicitly defining which pad a peripheral (such as an LPUART Rx line) should listen to when multiple routing options exist. The implemented `iomuxSetPadAltMode` driver automates this configuration process, ensuring accurate hardware routing and preventing signal collisions prior to initializing higher-level protocols.

The header file is attached here for quick referencing.

IOMUX.h

```
#ifndef IOMUX_H
#define IOMUX_H

#include <stdint.h>

#define IOMUXC_BASE 0x443C0000

/* Pad Mux Registers for P11 Expansion Header Pins */
#define MUX_REG_GPIO_IO14      (IOMUXC_BASE + 0x0048)
#define MUX_REG_GPIO_IO15      (IOMUXC_BASE + 0x004C)

#define ALT_MODE_LPUART4      6

/* DAISY Registers */
#define DAISY_REG_LPUART4_RX (IOMUXC_BASE + 0x047C)

/* DAISY Registers values*/
#define DAISY_VALUE_GPIO_IO15_LPUART4 0

void iomuxSetPadAltMode(uintptr_t mux_reg_address, uint8_t
    alt_mode, uint8_t sion);
#endif /* IOMUX_H */
```

4.2 Generic Interrupt Controller (GIC)

The Generic Interrupt Controller (GIC) is the nerve center for managing hardware interrupts on the ARM architecture. In our bare-metal environment, handling interrupts manually via raw registers is highly error-prone. To solve this, the `GIC.h` file defines the critical base addresses for the GIC Distributor and Redistributor, directly mapping the `GICD_CTLR` and `GICR_WAKER` registers.

We implemented HALs (Hardware Abstraction Layers) to configure the generic interrupt controller and set up the CPU Core Exception state, safely pointing the CPU to our custom vector table. Additionally, drivers for enabling specific interrupt IDs, acknowledging fired interrupts, and signaling the end of an interrupt (EOI) were created. This transforms a complex hardware requirement into a clean, easy-to-use API for routing external events to the processor.

The header file is attached here for quick referencing.

GIC.h

```
#ifndef _GIC_H
#define _GIC_H

#include <stdint.h>

#define GICD_BASE      0x48000000ULL // Base address for GIC
    Distributor
#define GICR_BASE      0x48040000ULL // Base address for GIC
    Redistributor
#define GICR_SGI_BASE  0x48050000ULL // Base address for GIC
    Redistributor SGI registers
#define GICD_CTLR      (*(volatile uint32_t*)(GICD_BASE + 0x0000
    )) // GIC Distributor Control Register
#define GICR_WAKER      (*(volatile uint32_t*)(GICR_BASE + 0x0014
    )) // GIC Redistributor WAKER Register

void gicINIT(void);
void gicCPUInit(uintptr_t vector_table_ptr);
void gicEnableInterrupt(uint32_t intid);
uint32_t gicAcknowledgeInterrupt(void);
void gicEndOfInterrupt(uint32_t iar);
#endif
```

4.3 General-Purpose Input/Output (GPIOs)

The GPIO.h file provides a comprehensive abstraction of the i.MX91's GPIO modules. By mapping the exact register layouts from the reference manual into the GPIO_TypeDef C structure, we eliminated the need to calculate register offsets manually. We mapped the base addresses for GPIO1 through GPIO4 and cast them to our structural pointers, creating instant access to any GPIO port.

We implemented drivers to initialize pin modes (Input/Output) and to read/write logical states by passing arguments to the HALs. Instead of dealing with direction registers (PDDR), output registers (PDOR), and input registers (PDIR) directly with bitwise logic, a developer can now simply call gpioWrite() or gpioRead().

The header file is attached here for quick referencing.

GPIO.h

```
#ifndef GPIO_H
#define GPIO_H

#include <stdint.h>

#define LOW  0
#define HIGH 1
```

```

typedef enum {
    INPUT_MODE = 0,
    OUTPUT_MODE
} GPIO_PinMode;

typedef struct {
    volatile uint32_t VERID;           /* 0x00 Version ID */
    volatile uint32_t PARAM;           /* 0x04 Parameter */
    uint32_t RESERVED_0;               /* 0x08 Padding */
    volatile uint32_t LOCK;            /* 0x0C Lock */
    volatile uint32_t PCNS;            /* 0x10 Pin Control Nonsecure */
    /*
    volatile uint32_t ICNS;            /* 0x14 Interrupt Control
    Nonsecure */
    volatile uint32_t PCNP;            /* 0x18 Pin Control
    Nonprivilege */
    volatile uint32_t ICNP;            /* 0x1C Interrupt Control
    Nonprivilege */
    uint32_t RESERVED_1[8];            /* 0x20 - 0x3C Padding */
    volatile uint32_t PDOR;            /* 0x40 Port Data Output */
    volatile uint32_t PSOR;            /* 0x44 Port Set Output */
    volatile uint32_t PCOR;            /* 0x48 Port Clear Output */
    volatile uint32_t PTOR;            /* 0x4C Port Toggle Output */
    volatile uint32_t PDIR;            /* 0x50 Port Data Input */
    volatile uint32_t PDDR;            /* 0x54 Port Data Direction */
    volatile uint32_t PIDR;            /* 0x58 Port Input Disable */
    uint32_t RESERVED_2;               /* 0x5C Padding */
    volatile uint8_t PnDR[32];         /* 0x60 - 0x7F Pin Data (PODR
    - P31DR) */
    volatile uint32_t ICR[32];         /* 0x80 - 0xFC Interrupt
    Control (ICR0 - ICR31) */
    volatile uint32_t GICLR;           /* 0x100 Global Interrupt
    Control Low */
    volatile uint32_t GICHR;           /* 0x104 Global Interrupt
    Control High */
    uint32_t RESERVED_3[6];            /* 0x108 - 0x11C Padding */
    volatile uint32_t ISFR[2];         /* 0x120 - 0x124 Interrupt
    Status Flag (ISFR0 - ISFR1) */
} GPIO_TypeDef;

#define GPIO1_BASE 0x47400000
#define GPIO2_BASE 0x43810000
#define GPIO3_BASE 0x43820000
#define GPIO4_BASE 0x43830000

#define GPIO1 ((GPIO_TypeDef *) GPIO1_BASE)
#define GPIO2 ((GPIO_TypeDef *) GPIO2_BASE)
#define GPIO3 ((GPIO_TypeDef *) GPIO3_BASE)
#define GPIO4 ((GPIO_TypeDef *) GPIO4_BASE)

```

```

void gpioPinINIT(GPIO_TypeDef *gpio, uint8_t pin, GPIO_PinMode
mode);
void gpioWrite(GPIO_TypeDef *gpio, uint8_t pin, uint8_t value);
uint8_t gpioRead(GPIO_TypeDef *gpio, uint8_t pin);
#endif /* GPIO_H */

```

4.4 Low-Power Universal Asynchronous Receiver-Transmitter (LPUART)

The LPUART.h file establishes a complete software interface for the i.MX91's Low-Power UART peripherals. We designed the LPUART_TypeDef structure to perfectly map the memory layout, including FIFO, baud rate, control, and status registers. The base addresses for all eight LPUART instances (LPUART1 through LPUART8) are defined and cast for immediate use.

Coming to the drivers, the lpuartINIT function calculates and configures the exact baud rate against the hardware's source clock frequency, while enabling transmitter and receiver pipelines. We also developed standard I/O functions—such as sending single characters, printing strings, formatting hexadecimal outputs, and handling both blocking and non-blocking character reception.

The header file is attached here for quick referencing.

LPUART.h

```

#ifndef LPUART_H
#define LPUART_H

#include <stdint.h>

typedef struct {
    volatile uint32_t VERID;           /* 0x00 Version ID */
    volatile uint32_t PARAM;           /* 0x04 Parameter */
    volatile uint32_t GLOBAL;          /* 0x08 Global */
    volatile uint32_t PINCFG;          /* 0x0C Pin Configuration */
    volatile uint32_t BAUD;             /* 0x10 Baud Rate */
    volatile uint32_t STAT;             /* 0x14 Status */
    volatile uint32_t CTRL;             /* 0x18 Control */
    volatile uint32_t DATA;            /* 0x1C Data */
    volatile uint32_t MATCH;            /* 0x20 Match Address */
    volatile uint32_t MODIR;            /* 0x24 Modem IrDA */
    volatile uint32_t FIFO;             /* 0x28 FIFO */
    volatile uint32_t WATER;            /* 0x2C Watermark */
    volatile uint32_t DATAR0;           /* 0x30 Data Read-Only */
    uint32_t RESERVED_0[3];             /* 0x34 - 0x3C Padding */
    volatile uint32_t MCR;              /* 0x40 Modem Control */
    volatile uint32_t MSR;              /* 0x44 Modem Status */
    volatile uint32_t REIR;             /* 0x48 Receiver Extended Idle */
}

```

```

    volatile uint32_t TEIR;          /* 0x4C Transmitter Extended
        Idle */
    volatile uint32_t HDCR;          /* 0x50 Half Duplex Control */
    uint32_t RESERVED_1;             /* 0x54 Padding */
    volatile uint32_t TOCR;          /* 0x58 Timeout Control */
    volatile uint32_t TOSR;          /* 0x5C Timeout Status */
    volatile uint32_t TIMEOUT[4];    /* 0x60 - 0x6C Timeout N (
        TIMEOUT0 - TIMEOUT3) */
    uint32_t RESERVED_2[100];        /* 0x70 - 0x1FC Padding */
    volatile uint32_t TCBR[128];     /* 0x200 - 0x3FC Transmit
        Command Burst (TCBR0 - TCBR127) */
    volatile uint32_t TDBR[256];     /* 0x400 - 0x7FC Transmit Data
        Burst (TDBR0 - TDBR255) */
} LPUART_TypeDef;

#define LPUART1_BASE 0x44380000
#define LPUART2_BASE 0x44390000
#define LPUART3_BASE 0x42570000
#define LPUART4_BASE 0x42580000
#define LPUART5_BASE 0x42590000
#define LPUART6_BASE 0x425A0000
#define LPUART7_BASE 0x42690000
#define LPUART8_BASE 0x426A0000

#define LPUART1 ((LPUART_TypeDef *) LPUART1_BASE)
#define LPUART2 ((LPUART_TypeDef *) LPUART2_BASE)
#define LPUART3 ((LPUART_TypeDef *) LPUART3_BASE)
#define LPUART4 ((LPUART_TypeDef *) LPUART4_BASE)
#define LPUART5 ((LPUART_TypeDef *) LPUART5_BASE)
#define LPUART6 ((LPUART_TypeDef *) LPUART6_BASE)
#define LPUART7 ((LPUART_TypeDef *) LPUART7_BASE)
#define LPUART8 ((LPUART_TypeDef *) LPUART8_BASE)

/* Bit Definitions */
#define LPUART_STAT_TDRE (1 << 23) /* Transmit Data Register
    Empty Flag */
#define LPUART_STAT_RDRF (1 << 21) /* Receive Data Register Full
    Flag */
#define LPUART_CTRL_TE (1 << 19) /* Transmitter Enable */
#define LPUART_CTRL_RE (1 << 18) /* Receiver Enable */
#define LPUART_FIFO_TXFLUSH (1 << 15) /* Transmit FIFO Flush */
#define LPUART_FIFO_RXFLUSH (1 << 14) /* Receive FIFO Flush */
#define LPUART_FIFO_TXFE (1 << 7) /* Transmit FIFO Enable */
#define LPUART_FIFO_RXFE (1 << 3) /* Receive FIFO Enable */

void lpuartPutChar(LPUART_TypeDef *lpuart, char c);
void lpuartPrintString(LPUART_TypeDef *lpuart, const char *str);
void lpuartPrintHex(LPUART_TypeDef *lpuart, uint32_t val);
char lpuartGetCharBlocking(LPUART_TypeDef *lpuart);
char lpuartGetCharNonBlocking(LPUART_TypeDef *lpuart);

```

```
void lpuartINIT(LPUART_TypeDef *lpuart, uint32_t baudrate,
    uint32_t src_clock_hz);
#endif /* LPUART_H */
```

4.5 System Counter (SYS_CTR)

The `SYS_CTR.h` file leverages the 64-bit ARM Generic Timer to provide extremely precise, deterministic timing constraints directly from the CPU core.

The functions `sysctrGetTicks` and `sysctrGetFreq` interact directly with the CPU's system registers (`CNTPCT_ELO` and `CNTFRQ_ELO`) to establish timing baselines. Leveraging these counts, we built `sysctrDelay_us` and `sysctrDelay_ms` functions for calculating exact microsecond and millisecond intervals, providing the fundamental delays necessary for stabilizing sensors, displays, and synchronous protocols on the bare-metal OS.

The header file is attached here for quick referencing.

LPUART.h

```
#ifndef SYS_CTR_H
#define SYS_CTR_H

#include <stdint.h>

uint64_t sysctrGetTicks(void);
uint32_t sysctrGetFreq(void);
void sysctrDelay_us(uint32_t us);
void sysctrDelay_ms(uint32_t ms);
#endif /* SYS_CTR_H */
```


Chapter 5

Interfacing with the Physical World: HC-SR04 Sonar

5.1 Hardware Wiring

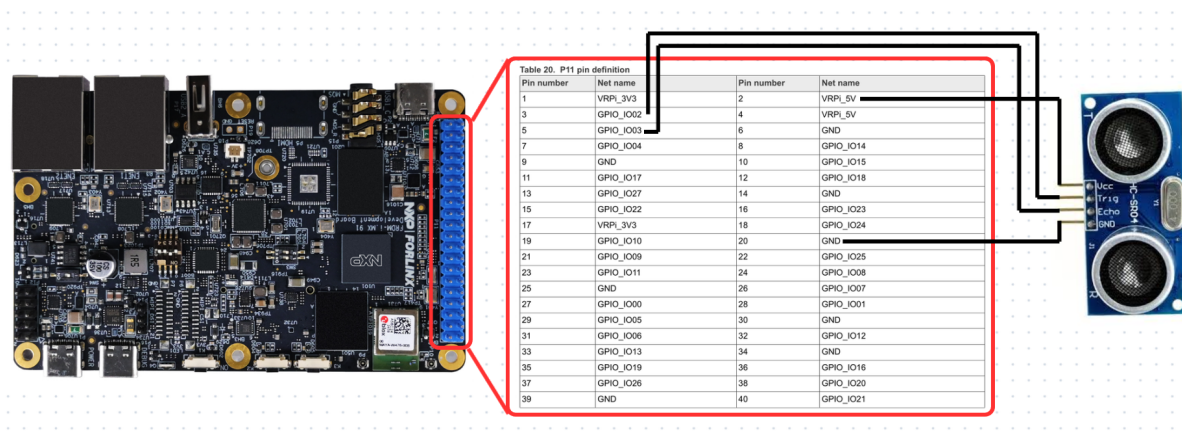


Figure 5.1: Connection of NXP with ultrasonic sensor for our code

5.2 Steps to Run

To compile and execute the bare-metal sonar driver, follow these steps from your Ubuntu host machine:

1. **Physical Connection:** Ensure the HC-SR04 sensor is securely wired to the FRDM-i.MX91 GPIO pins exactly as illustrated in Figure 5.1.
2. **Navigate to Project Root:** Open your terminal and navigate to the root directory of the BareMetal_FRDM_IMX91 repository.
3. **Compile the Application:** Use the master build system to compile the `sonar_proximity` app. This will link the generic GPIO and Timer drivers with your sonar logic.

```
make APP=sonar_proximity
```

4. **Deploy to the Board (UMS Method):** Run the installation macro to print the deployment paths:

```
make usb_ums_install APP=sonar_proximity
```

Based on your storage medium, follow the corresponding flow below.

Option A: SD Card Deployment

- (a) In the U-Boot serial console, expose the SD card partition to your PC:

```
ums 0 mmc 1:3
```

- (b) On your Ubuntu host terminal, copy the newly compiled binary to the mounted SD card:

```
cp Apps/sonar_proximity/build/sonar_proximity.bin /media/shreekar/BM_PROGRAMS/
```

- (c) Eject the BM_PROGRAMS drive from your Ubuntu file manager, in the U-Boot terminal UMS will automatically stop after ejecting.
- (d) Load the binary into RAM, flush the caches, and execute:

```
fatload mmc 1:3 0x80000000 sonar_proximity.bin && dcache  
flush && icache flush && go 0x80000000
```

Option B: eMMC Deployment

- (a) In the U-Boot serial console, expose the onboard eMMC to your PC:

```
ums 0 mmc 0
```

- (b) On your Ubuntu host, copy Apps/sonar_proximity/build/sonar_proximity.bin to the newly mounted 'boot' drive.
- (c) Eject the drive from your Ubuntu file manager, then press Ctrl+C in the U-Boot terminal to stop UMS mode.
- (d) Load the binary into RAM, flush the caches, and execute:

```
fatload mmc 0:1 0x80000000 sonar_proximity.bin && dcache  
flush && icache flush && go 0x80000000
```

5. **Monitor Output:** Once U-Boot hands control over to your bare-metal code, open your serial terminal to view the real-time distance calculations (in centimeters) printed by the board.

```
sudo picocom -b 115200 /dev/ttyUSB0
```